OPERATING SYSTEMS

Secure File Vault using System Calls in C

22AIE201

Group-12

Lakshmi Tejaswi-24116

Kiranmai-24119

Sudeshna-24120

Introduction

- Secure File Vault is a file encryption and management system.
- Provides encryption/decryption, hide/unhide, and secure delete.
- Stores user login credentials securely in a .db file.
- Designed for Ubuntu/Linux systems with system-level file operations.

Problem Statement

- Sensitive files often lack proper security on local machines.
- Manual file encryption and deletion is time-consuming and error-prone.
- Users need secure
- Solution: An automated vault that encrypts, hides, deletes, and manages permissions efficiently.

OBJECTIVES

- Secure File Storage: Encrypt and decrypt files to protect sensitive data.
- User Authentication: Login/registration with persistent .db storage.
- File Privacy Management: Hide/unhide files using rename() + chmod().
- Secure File Deletion: Overwrite and delete files with unlink().
- System-Level Control: Demonstrate use of system calls (open, read, write, close, chmod, rename, unlink).

Tools & Technology

- Programming Language: C (Linux environment)
- Compiler: gcc
- System Calls Used: open(), read(), write(), close(), unlink(), chmod(), rename()
- Database: Simple .db file for user login storage
- Editor: Nano in Ubuntu terminal

System Design

- Encryption/Decryption Module: XOR-based encryption with chunked read/write.
- Hide/Unhide Module: Change filename and permissions.
- Secure Delete Module: Overwrite + unlink().
- Permissions Manager: Controls file accessibility.

Implementation Details

- Login System: Checks or registers user in users.db.
- File Operations: Chunked read/write with XOR encryption.
- File Management: Hide/unhide (rename() + chmod()) and secure delete.
- Logout Feature: Allows switching users without restarting program.

System Calls Used

open() – Opening a File

- Purpose: Used to open a file so that your program can read from it or write to it.

read() – Reading File Content

- Purpose: Reads the contents of a file into memory (like loading text into a buffer).

write() – Writing to a File

- Purpose: Writes data from memory into a file.

close() – Closing a File

- Purpose: Releases the file descriptor when you're done.

unlink() – Deleting a File

- Purpose: Deletes a file from the filesystem.

chmod() – Changing File Permissions

- Purpose: Changes who can read, write, or execute a file.

rename()- hide/unhide files

Workflow

- Check User Database → Register if no users.
- Login Screen → Enter username & password.
- Vault Menu → Encrypt, Decrypt, Hide, Unhide, Secure Delete, Logout.
- File Output → Files encrypted/decrypted, hidden/unhidden, or deleted.
- Logout → Return to login for next user.

Output Demo

- ser Registration: .db file created, stores login credentials.
- Login Success/Failure: Grants access to registered users.
- File Encryption/Decryption: Original files encrypted/decrypted correctly.
- Hide/Unhide File: Files become hidden/unhidden with correct permissions.
- Secure Delete: Files overwritten and deleted permanently.

Expected Results

```
Login:
Username: kiranmai
Password: kiranmai
Access Granted! Welcome kiranmai

========== Secure File Vault ==========
1. Encrypt file
2. Decrypt file
3. Hide file
4. Unhide file
5. Secure delete
6. Logout
Enter choice: 1
Enter file to encrypt: d.c
Processed file: d.c
```

Conclusion

- Provides encryption, hiding, secure deletion, and permission management.
- Login system ensures controlled access.
- Fully implemented using Linux system calls, demonstrating low-level file management skills.

THANK YOU